

Aim :

To find the contiguous subarray within a one-dimensional integer array nums that has the largest sum, and return that maximum sum.

Algorithm :**1. Initialize:**

- Set currentSum = nums[0] to track the maximum sum ending at the current index.
- Set maxSum = nums[0] to track the global maximum subarray sum found so far.

2. Iterate:

- Loop through the array from index 1 to nums.length - 1.
- For each element nums[i]:
 - Update currentSum = Math.max(nums[i], currentSum + nums[i]). This chooses whether to start a new subarray at nums[i] or extend the existing subarray.
 - Update maxSum = Math.max(maxSum, currentSum).

3. Terminate & Return:

- After the loop finishes, return maxSum.

Program :

```
import java.util.Scanner;
```

```
public class MaximumSubarray {
```

```
    public static int maxSubArray(int[] nums) {
```

```
        int currentSum = nums[0];
```

```
int maxSum = nums[0];

for (int i = 1; i < nums.length; i++) {
    currentSum = Math.max(nums[i], currentSum + nums[i]);
    maxSum = Math.max(maxSum, currentSum);
}

return maxSum;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    if (!sc.hasNextInt()) return;

    int n = sc.nextInt();
    int[] nums = new int[n];
    for (int i = 0; i < n; i++) {
        nums[i] = sc.nextInt();
    }

    int result = maxSubArray(nums);
    System.out.println(result);

    sc.close();
}
}
```

Sample Inputs & Outputs :

Sample 1 :

Plaintext

Input:

9

-2 1 -3 4 -1 2 1 -5 4

Output:

6

Sample 2 :

Plaintext

Input:

1

1

Output:

1

Sample 3 :

Plaintext

Input:

5

5 4 -1 7 8

Output: 23

Aim :

To determine the number of contiguous segments in a chocolate bar array s of length m whose elements sum up to Ron's birth day d .

Algorithm :

1. **Length Validation:** Let n be the total number of elements in array s . If $n < m$, return 0.
2. **Initial Segment Sum:** Compute the sum of the first m elements (indices 0 to $m - 1$).
3. **Initialize Counter:** Set count = 1 if this sum equals d , otherwise set count = 0.
4. **Sliding Window:**
 - Iterate index i from m to $n - 1$.
 - Update the running sum by subtracting the outgoing element $s[i - m]$ and adding the incoming element $s[i]$.
 - If the running sum equals d , increment count by 1.
5. **Output Result:** Return the final value of count.

Program :

Java

```
import java.io.*;
```

```
import java.util.*;
```

```
public class Solution {
```

```
    // Complete the birthday function below
```

```
    public static int birthday(List<Integer> s, int d, int m) {
```

```
        int n = s.size();
```

```
        if (n < m) {
```

```
            return 0;
```

```
        }
```

```

int count = 0;

int currentSum = 0;

// Sum of the first window of length m
for (int i = 0; i < m; i++) {
    currentSum += s.get(i);
}

if (currentSum == d) {
    count++;
}

// Slide the window across the remaining elements
for (int i = m; i < n; i++) {
    currentSum += s.get(i) - s.get(i - m);
    if (currentSum == d) {
        count++;
    }
}

return count;
}

public static void main(String[] args) throws IOException {
    BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

    // Read size of array

```

```

int n = Integer.parseInt(bufferedReader.readLine().trim());

// Read array elements
String[] sTemp = bufferedReader.readLine().trim().split("\\s+");
List<Integer> s = new ArrayList<>();
for (int i = 0; i < n; i++) {
    s.add(Integer.parseInt(sTemp[i]));
}

// Read d and m
String[] dm = bufferedReader.readLine().trim().split("\\s+");
int d = Integer.parseInt(dm[0]);
int m = Integer.parseInt(dm[1]);

int result = birthday(s, d, m);
System.out.println(result);

bufferedReader.close();
}
}

```

Sample Inputs & Outputs :

Sample 0 :

Plaintext

Input:

5

1 2 1 3 2

3 2

Output:

2

Sample 1 :

Plaintext

Input:

6

1 1 1 1 1 1

3 2

Output:

0

Sample 2 :

Plaintext

Input:

1

4

4 1

Output:

1

AIM :

To write a Java program that takes an array of integers as input and finds the maximum possible sum of a contiguous subarray and the maximum possible sum of a subsequence.

ALGORITHM :

Step 1: Start the program. **Step 2:** Read the number of test cases t. **Step 3:** For each test case, read the size of the array n and then read the n integer elements into an array. **Step 4:** If the array is empty, return 0 for both sums. Otherwise, proceed to Step 5. **Step 5:** Initialize variables for the subarray calculation: set both maxSubarray and currentSubarray to the first element of the array. **Step 6:** Initialize variables for the subsequence calculation: set maxSubsequence to 0, maxElement to the first element, and a boolean flag hasPositive to false. **Step 7:** Loop through the array from the first element to the last element:

- **7a.** Update currentSubarray to the maximum of the *current element* and (*currentSubarray + current element*).
- **7b.** Update maxSubarray to the maximum of *maxSubarray* and *currentSubarray*.
- **7c.** Update maxElement to keep track of the highest single number in the array.
- **7d.** If the current element is greater than 0, add it to maxSubsequence and set hasPositive to true. **Step 8:** After the loop finishes, check if hasPositive is false (which means all numbers were negative). If true, set maxSubsequence equal to maxElement. **Step 9:** Print maxSubarray and maxSubsequence separated by a space. **Step 10:** Repeat Steps 3-9 for all test cases. **Step 11:** Stop the program.

PROGRAM :

Java

```
import java.util.Scanner;
```

```
public class MaxSubarrayAndSubsequence {
```

```
    public static void findMaximumSums(int[] arr) {
```

```
        if (arr.length == 0) {
```

```
        System.out.println("0 0");  
        return;  
    }  
}
```

```
int maxSubarray = arr[0];  
int currentSubarray = arr[0];
```

```
int maxSubsequence = 0;  
int maxElement = arr[0];  
boolean hasPositive = false;
```

```
for (int i = 0; i < arr.length; i++) {  
    int val = arr[i];
```

```
    if (i > 0) {  
        currentSubarray = Math.max(val, currentSubarray + val);  
        maxSubarray = Math.max(maxSubarray, currentSubarray);  
    }  
}
```

```
maxElement = Math.max(maxElement, val);  
if (val > 0) {  
    maxSubsequence += val;  
    hasPositive = true;  
}  
}
```

```
if (!hasPositive) {  
    maxSubsequence = maxElement;
```

```

    }

    System.out.println(maxSubarray + " " + maxSubsequence);
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.println("Enter number of test cases:");
    if (scanner.hasNextInt()) {
        int t = scanner.nextInt();

        for (int i = 0; i < t; i++) {
            System.out.println("Enter size of array for test case " + (i + 1) + ":");
            int n = scanner.nextInt();
            int[] arr = new int[n];

            System.out.println("Enter " + n + " elements:");
            for (int j = 0; j < n; j++) {
                arr[j] = scanner.nextInt();
            }

            System.out.print("Output: ");
            findMaximumSums(arr);
        }
    }
    scanner.close();
}

```

}

OUTPUT :

Test Run 1 (Sample Input 0):

Plaintext

Enter number of test cases:

2

Enter size of array for test case 1:

4

Enter 4 elements:

1 2 3 4

Output: 10 10

Enter size of array for test case 2:

6

Enter 6 elements:

2 -1 2 3 4 -5

Output: 10 11

Test Run 2 (Sample Input 1):

Plaintext

Enter number of test cases:

1

Enter size of array for test case 1:

5

Enter 5 elements:

-2 -3 -1 -4 -6

Output: -1 -1

AIM :

To write a Java program that finds the maximum possible sum of a non-empty subarray within a circular integer array.

ALGORITHM :

Step 1: Start the program. **Step 2:** Initialize totalSum to 0. Initialize currentMax to 0 and maxSum to the lowest possible integer value. Initialize currentMin to 0 and minSum to the highest possible integer value. **Step 3:** Iterate through each element in the array:

- **3a.** Add the current element to totalSum.
- **3b.** Update currentMax to the maximum of the current element itself or the sum of currentMax and the current element.
- **3c.** Update maxSum to the maximum of maxSum and currentMax.
- **3d.** Update currentMin to the minimum of the current element itself or the sum of currentMin and the current element.
- **3e.** Update minSum to the minimum of minSum and currentMin. **Step 4:** After the loop, check if maxSum is less than 0. If it is, this means all elements in the array are negative, so return maxSum directly. **Step 5:** If maxSum is greater than or equal to 0, return the maximum between maxSum (the maximum non-wrapping subarray) and totalSum - minSum (the maximum wrapping subarray). **Step 6:** Stop the program.

PROGRAM :

```
public class CircularSubarraySum {  
  
    public static int maxSubarraySumCircular(int[] nums) {  
        int totalSum = 0;  
        int currentMax = 0;  
        int maxSum = Integer.MIN_VALUE;  
        int currentMin = 0;  
        int minSum = Integer.MAX_VALUE;
```

```

    for (int num : nums) {
        totalSum += num;

        currentMax = Math.max(currentMax + num, num);
        maxSum = Math.max(maxSum, currentMax);

        currentMin = Math.min(currentMin + num, num);
        minSum = Math.min(minSum, currentMin);
    }

    if (maxSum < 0) {
        return maxSum;
    }

    return Math.max(maxSum, totalSum - minSum);
}

public static void main(String[] args) {
    int[] nums1 = {1, -2, 3, -2};
    System.out.println("Input: nums = [1, -2, 3, -2]");
    System.out.println("Output: " + maxSubarraySumCircular(nums1));
    System.out.println();

    int[] nums2 = {5, -3, 5};
    System.out.println("Input: nums = [5, -3, 5]");
    System.out.println("Output: " + maxSubarraySumCircular(nums2));
    System.out.println();
}

```

```
int[] nums3 = {-3, -2, -3};  
System.out.println("Input: nums = [-3, -2, -3]");  
System.out.println("Output: " + maxSubarraySumCircular(nums3));  
}  
}
```

OUTPUT :

Plaintext

Input: nums = [1, -2, 3, -2]

Output: 3

Input: nums = [5, -3, 5]

Output: 10

Input: nums = [-3, -2, -3]

Output: -2

AIM :

To write a Java program that implements the myAtoi(string s) function to convert a string to a 32-bit signed integer, handling leading whitespaces, sign characters, non-digit interruptions, and 32-bit integer overflow/underflow.

ALGORITHM :

Step 1: Start the program. **Step 2:** Check if the input string is empty or null. If so, return 0. **Step 3:** Initialize an index i to 0, an integer sign to 1, and an integer result to 0. **Step 4:** Iterate and increment i to skip any leading whitespace (' ') characters. **Step 5:** If i is within the string bounds, check if the character at i is '+' or '-'. If it is '-', set sign to -1. If it is either, increment i by 1. **Step 6:** Loop through the remaining characters as long as i is within bounds and the character at i is a digit (between '0' and '9'):

- **6a.** Convert the character to its integer digit value.
- **6b.** Check for integer overflow. If result is greater than Integer.MAX_VALUE / 10, or if it is exactly Integer.MAX_VALUE / 10 but the current digit is greater than 7 (since Integer.MAX_VALUE ends in 7), it means an overflow will occur.
- **6c.** If an overflow is detected, return Integer.MAX_VALUE if sign is 1, or Integer.MIN_VALUE if sign is -1.
- **6d.** Multiply result by 10 and add the current digit. Increment i. **Step 7:** After the loop, return the result multiplied by the sign. **Step 8:** Stop the program.

PROGRAM :

```
public class StringToIntegerAtoi {  
  
    public static int myAtoi(String s) {  
        int i = 0;  
        int n = s.length();  
  
        while (i < n && s.charAt(i) == ' ') {  
            i++;  
        }  
    }  
}
```

```

    }

    if (i >= n) {
        return 0;
    }

    int sign = 1;
    if (s.charAt(i) == '+' || s.charAt(i) == '-') {
        sign = (s.charAt(i) == '-') ? -1 : 1;
        i++;
    }

    int result = 0;
    while (i < n && s.charAt(i) >= '0' && s.charAt(i) <= '9') {
        int digit = s.charAt(i) - '0';

        if (result > Integer.MAX_VALUE / 10 ||
            (result == Integer.MAX_VALUE / 10 && digit > Integer.MAX_VALUE % 10)) {
            return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
        }

        result = result * 10 + digit;
        i++;
    }

    return result * sign;
}

```

```
public static void main(String[] args) {  
    System.out.println("Input: s = \"42\"");  
    System.out.println("Output: " + myAtoi("42"));  
    System.out.println();  
  
    System.out.println("Input: s = \"-042\"");  
    System.out.println("Output: " + myAtoi("-042"));  
    System.out.println();  
  
    System.out.println("Input: s = \"1337c0d3\"");  
    System.out.println("Output: " + myAtoi("1337c0d3"));  
    System.out.println();  
  
    System.out.println("Input: s = \"0-1\"");  
    System.out.println("Output: " + myAtoi("0-1"));  
    System.out.println();  
  
    System.out.println("Input: s = \"words and 987\"");  
    System.out.println("Output: " + myAtoi("words and 987"));  
}  
}
```

OUTPUT :

Plaintext

Input: s = "42"

Output: 42

Input: s = "-042"

Output: -42

Input: s = "1337c0d3"

Output: 1337

Input: s = "0-1"

Output: 0

Input: s = "words and 987"

Output: 0

AIM :

To write a Java program that calculates the minimum number of character deletions required to change a string containing only the characters 'A' and 'B' so that there are no matching adjacent characters.

ALGORITHM :

Step 1: Start the program. **Step 2:** Read the integer q which represents the number of test queries. **Step 3:** Loop q times to process each query string s. **Step 4:** For each string, initialize an integer variable deletions to 0. **Step 5:** Iterate through the string from the index 0 up to length - 2 (the second-to-last character). **Step 6:** Inside the loop, compare the character at the current index i with the character at the next index i + 1. **Step 7:** If the two adjacent characters are identical, increment the deletions counter by 1. This mimics the process of deleting one of the matching adjacent characters. **Step 8:** After the loop finishes checking the entire string, return or print the deletions count. **Step 9:** Repeat Steps 4-8 for all q strings. **Step 10:** Stop the program.

PROGRAM :

```
import java.util.Scanner;

public class AlternatingCharacters {

    public static int alternatingCharacters(String s) {

        int deletions = 0;

        for (int i = 0; i < s.length() - 1; i++) {

            if (s.charAt(i) == s.charAt(i + 1)) {

                deletions++;

            }

        }

    }

}
```

```
        return deletions;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter number of queries:");
        if (scanner.hasNextInt()) {
            int q = scanner.nextInt();

            System.out.println("Enter the strings:");
            for (int i = 0; i < q; i++) {
                String s = scanner.next();
                int result = alternatingCharacters(s);
                System.out.println("Output: " + result);
            }
        }

        scanner.close();
    }
}
```

OUTPUT :

Test Run (Based on Sample Input):

Plaintext

Enter number of queries:

5

Enter the strings:

AAAA

Output: 3

BBBBB

Output: 4

ABABABAB

Output: 0

BABABA

Output: 0

AAABBB

Output: 4

AIM :

To write a Java program that finds the length of the longest substring without repeating characters in a given string.

ALGORITHM :

Step 1: Start the program. **Step 2:** Define a method that takes a string *s* as input. If the string is null or empty, return 0. **Step 3:** Initialize a variable *maxLength* to 0 to keep track of the maximum substring length found so far. **Step 4:** Initialize an integer array of size 128 (to cover ASCII characters) to store the most recently seen index of each character. **Step 5:** Initialize a left pointer to 0, representing the start of the current sliding window without duplicates. **Step 6:** Use a right pointer in a loop to traverse each character of the string from index 0 to the end. **Step 7:** Inside the loop, check the current character. Update the left pointer to the maximum of its current value or the previously stored index of this character. This ensures the window shrinks just enough to exclude the duplicate. **Step 8:** Update the array with the current character's new position (*right* + 1). **Step 9:** Calculate the current window length (*right* - *left* + 1) and update *maxLength* if the current length is greater. **Step 10:** After the loop completes, return *maxLength*. **Step 11:** Stop the program.

PROGRAM :

Java

```
public class LongestSubstring {

    public static int lengthOfLongestSubstring(String s) {

        int maxLength = 0;

        int[] charIndex = new int[128];

        int left = 0;

        for (int right = 0; right < s.length(); right++) {

            char currentChar = s.charAt(right);

            left = Math.max(left, charIndex[currentChar]);

        }

    }

}
```

```

        charIndex[currentChar] = right + 1;

        maxLength = Math.max(maxLength, right - left + 1);
    }

    return maxLength;
}

public static void main(String[] args) {
    System.out.println("Input: s = \"abcabcbb\"");
    System.out.println("Output: " + lengthOfLongestSubstring("abcabcbb"));
    System.out.println();

    System.out.println("Input: s = \"bbbbbb\"");
    System.out.println("Output: " + lengthOfLongestSubstring("bbbbbb"));
    System.out.println();

    System.out.println("Input: s = \"pwwkew\"");
    System.out.println("Output: " + lengthOfLongestSubstring("pwwkew"));
}
}

```

OUTPUT :

Plaintext

Input: s = "abcabcbb"

Output: 3

Input: s = "bbbbbb"

Output: 1

Input: s = "pwwkew"

Output: 3

AIM :

To write a Java program that takes a list of strings and a pattern, and returns a list of words that match the pattern based on a character bijection (permutation of letters).

ALGORITHM :

Step 1: Start the program. **Step 2:** Define a method `findAndReplacePattern` that takes an array of strings `words` and a string `pattern` as input. **Step 3:** Initialize an empty list of strings named `result` to store the matching words. **Step 4:** Iterate through each string `word` in the `words` array. **Step 5:** For each `word`, use a helper method `isMatch(word, pattern)` to check if it follows the same pattern. **Step 6:** In the `isMatch` method, initialize two integer arrays of size 26, `map1` and `map2`, to keep track of character mappings for the word and the pattern respectively. **Step 7:** Loop through the characters of the word and pattern simultaneously (since they are of the same length). **Step 8:** For each character at index `i`, check if the recorded index in `map1` for the word's character equals the recorded index in `map2` for the pattern's character.

- **8a.** If they are not equal, the bijection is broken. Return false.
- **8b.** If they are equal, update both `map1` and `map2` at the respective character's position with the value `i + 1` (to distinguish from the default 0). **Step 9:** If the loop in `isMatch` completes without returning false, return true. **Step 10:** Back in the main method, if `isMatch` returns true, add the word to the result list. **Step 11:** After iterating through all words, return the result list. **Step 12:** Stop the program.

PROGRAM :

```
import java.util.ArrayList;
import java.util.List;

public class FindAndReplacePattern {

    public static List<String> findAndReplacePattern(String[] words, String pattern) {
        List<String> result = new ArrayList<>();

        for (String word : words) {
            if (isMatch(word, pattern)) {
```

```

        result.add(word);
    }
}

return result;
}

private static boolean isMatch(String word, String pattern) {
    int[] map1 = new int[26];
    int[] map2 = new int[26];

    for (int i = 0; i < word.length(); i++) {
        int wChar = word.charAt(i) - 'a';
        int pChar = pattern.charAt(i) - 'a';

        if (map1[wChar] != map2[pChar]) {
            return false;
        }

        map1[wChar] = i + 1;
        map2[pChar] = i + 1;
    }

    return true;
}

public static void main(String[] args) {
    String[] words1 = {"abc", "deq", "mee", "aqq", "dkd", "ccc"};

```

```

String pattern1 = "abb";

System.out.println("Input: words = [\"abc\", \"deq\", \"mee\", \"aqq\", \"dkd\", \"ccc\"],
pattern = \"abb\"");

System.out.println("Output: " + findAndReplacePattern(words1, pattern1));

System.out.println();

String[] words2 = {"a", "b", "c"};

String pattern2 = "a";

System.out.println("Input: words = [\"a\", \"b\", \"c\"], pattern = \"a\"");

System.out.println("Output: " + findAndReplacePattern(words2, pattern2));

}

}

```

OUTPUT :

Plaintext

Input: words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb"

Output: [mee, aqq]

Input: words = ["a","b","c"], pattern = "a"

Output: [a, b, c]

AIM :

To write a Java program that takes an array of unique strings and returns all strings that are a substring of another string in the array.

ALGORITHM :

Step 1: Start the program. **Step 2:** Define a method stringMatching that takes an array of strings words as input. **Step 3:** Initialize an empty list of strings called result to store the substrings. **Step 4:** Start an outer loop to iterate through each string in the array using an index i. **Step 5:** Start an inner loop to iterate through the array again using an index j to compare the current string words[i] against all other strings words[j]. **Step 6:** Check if i is not equal to j (to avoid comparing a word with itself). **Step 7:** If they are not the same word, check if words[j] contains words[i] using the built-in contains() method. **Step 8:** If words[j] contains words[i], add words[i] to the result list and break out of the inner loop (since finding it as a substring once is sufficient). **Step 9:** After the outer loop completes, return the result list. **Step 10:** Stop the program.

PROGRAM :

```
import java.util.ArrayList;

import java.util.List;

public class StringMatchingInArray {

    public static List<String> stringMatching(String[] words) {

        List<String> result = new ArrayList<>();

        for (int i = 0; i < words.length; i++) {

            for (int j = 0; j < words.length; j++) {

                if (i != j && words[j].contains(words[i])) {

                    result.add(words[i]);

                    break;

                }

            }

        }

    }

}
```

```

    }

    return result;
}

public static void main(String[] args) {
    String[] words1 = {"mass", "as", "hero", "superhero"};
    System.out.println("Input: words = ["mass","as","hero","superhero"]);
    System.out.println("Output: " + stringMatching(words1));
    System.out.println();

    String[] words2 = {"leetcode", "et", "code"};
    System.out.println("Input: words = ["leetcode","et","code"]);
    System.out.println("Output: " + stringMatching(words2));
    System.out.println();

    String[] words3 = {"blue", "green", "bu"};
    System.out.println("Input: words = ["blue","green","bu"]);
    System.out.println("Output: " + stringMatching(words3));
}
}

```

OUTPUT :

Plaintext

Input: words = ["mass","as","hero","superhero"]

Output: [as, hero]

Input: words = ["leetcode","et","code"]

Output: [et, code]

Input: words = ["blue","green","bu"]

Output: []

AIM :

To write a Java program that implements the naive algorithm for pattern searching to find all starting indices where a given pattern occurs in a text string.

ALGORITHM :

Step 1: Start the program.

Step 2: Define a method searchPattern that takes two strings, text and pattern, as input.

Step 3: Initialize an empty list of integers called result to store the starting indices of the matches.

Step 4: Calculate the length of the text (n) and the length of the pattern (m).

Step 5: Use a for loop to iterate through the text from index $i = 0$ to $i = n - m$. This sets the starting point of the sliding window.

Step 6: Inside the loop, declare an integer j for the inner loop counter.

Step 7: Use an inner for loop to iterate from $j = 0$ to $m - 1$.

Step 8: Check if the character at `text.charAt(i + j)` is not equal to `pattern.charAt(j)`. If they do not match, break the inner loop.

Step 9: If the inner loop finishes without breaking (which means j is exactly equal to m), it implies a complete match was found. Add the index i to the result list.

Step 10: After the outer loop completes testing all possible starting indices, return the result list.

Step 11: Stop the program.

PROGRAM :

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class NaivePatternSearch {
```

```
    public static List<Integer> searchPattern(String text, String pattern) {
```

```
        List<Integer> result = new ArrayList<>();
```

```

int n = text.length();

int m = pattern.length();

for (int i = 0; i <= n - m; i++) {

    int j;

    for (j = 0; j < m; j++) {

        if (text.charAt(i + j) != pattern.charAt(j)) {

            break;

        }

    }

    if (j == m) {

        result.add(i);

    }

}

return result;

}

public static void main(String[] args) {

    String text1 = "geeksforgeeks";

    String pattern1 = "geeks";

    System.out.println("Input: text = \"" + text1 + "\", pattern = \"" + pattern1 + "\"");

    System.out.println("Output: " + searchPattern(text1, pattern1));

    System.out.println();

    String text2 = "aabaacaadaabaaba";

```

```
String pattern2 = "aaba";

System.out.println("Input: text = \"" + text2 + "\", pattern = \"" + pattern2 + "\"");

System.out.println("Output: " + searchPattern(text2, pattern2));

}

}
```

OUTPUT :

Plaintext

Input: text = "geeksforgeeks", pattern = "geeks"

Output: [0, 8]

Input: text = "aabaacaadaabaaba", pattern = "aaba"

Output: [0, 9, 12]